

ksformat :: Cheat Sheet

SAS-style PROC FORMAT for R — value→label mappings, numeric ranges, reverse formatting (invalue), date/time/datetime, expression labels, multilabel & more

INSTALL & LOAD

```
# From CRAN
install.packages("ksformat")
# From GitHub (dev)
remotes::install_github("crow16384/ksformat")
library(ksformat)
```

FORMAT CREATION

FNEW() — DISCRETE VALUE→LABEL VALUE

fnew(..., name=NULL, type="auto", default=NULL, multilabel=FALSE, ignore_case=FALSE)
... = named pairs key="label" | .missing/other = special labels | name = register in library

```
fnew(
  "M" = "Male",
  "F" = "Female",
  .missing = "Unknown",
  .other = "Other Gender",
  name = "sex"
)
```

SEX CHAR

M - Male
F - Female
.missing - Unknown
.other - Other Gender

FINPUT() — REVERSE (LABEL→VALUE) INVALUE

finput(..., name=NULL, target_type="numeric", missing_value=NA, ignore_case=FALSE)
... = named pairs label=VALUE | target_type = "numeric" | "character" output type

```
finput(
  "Male" = 1,
  "Female" = 2,
  name = "sex_inv"
)
```

SEX_INV INVALUE → NUM

Male - 1
Female - 2

FNEW_BID() — BIDIRECTIONAL VALUE + INVALUE

fnew_bid(..., name=NULL, type="auto", ignore_case=FALSE)
Creates both a VALUE format and matching _inv INVALUE simultaneously

```
fnew_bid(
  "A" = "Active",
  "I" = "Inactive",
  name = "status"
)
```

Creates both:
"status" VALUE format
"status_inv" INVALUE

FNEW_DATE() — DATE/TIME/DATETIME FORMAT

fnew_date(pattern, name=NULL, type="auto", .missing=NULL)
pattern = SAS format name (DATE9.) or R strftime pattern (%d.%m.%Y)

```
fnew_date("DATE9.", name="bday")
fnew_date("%d.%m.%Y",
  name="ru", type="date")
fnew_date("MMDDYY10.",
  name="us", .missing="No DATE")
```

bday DATE

Pattern: %d.%m.%Y (DATE9.)
SAS names auto-resolved; R patterns also supported

OPTIONS FOR FNEW()

PARAM	DEFAULT	WHAT IT DOES
name	NULL	Register in format library
type	"auto"	"numeric"/"character" key type
multilabel	FALSE	Allow multiple labels per value
ignore_case	FALSE	Case-insensitive matching
.missing	—	Label for NA/NaN/""
.other	—	Fallback for unmatched values

FORMAT APPLICATION

FPUT() — APPLY FORMAT (GENERIC)

fput(x, format, ..., keep_na=FALSE)
x = values | format = name or object | ... = extra args for expression labels

```
fput(
  c("M", "F", "M", NA, "X"),
  "sex"
)
```

"Male" "Female" "Male" "Unknown"
"Other Gender"

FPUTN() FPUTC() — TYPED APPLY

fputn(x, format_name, ...) · fputc(x, format_name, ...)
Numeric/character shorthand — format_name is always a string name

```
fputn(c(5, 30, 70), "age")
fputc(c("M", "F"), "sex")
```

"Child" "Adult" "Senior"
"Male" "Female"

FPUTK() — COMPOSITE KEY APPLY

fputk(..., format, sep="|", keep_na=FALSE)
... = vectors pasted into composite key (e.g. SUBJ|VISIT) before lookup

```
fnew("A1"="2025-01-15",
  "A2"="2025-02-20",
  "B1"="2025-03-10",
  .other="NOT FOUND",
  name="vd")
fputk(c("A", "A", "B", "B"),
  c(1,2,1,3), format="vd")
```

"2025-01-15" "2025-02-20" "2025-03-10"
"NOT FOUND"

FPUTK() + FMAP() / FPARSE() — DATE LOOKUP

Data-driven from data frame
fnew(fmap(paste(SV\$SUBJ, SV\$VISIT, sep="|"), as.Date(SV\$DTC)), name="svdtn", type="Date")
Or via fparse() text:
VALUE svdtn (Date)
format: %Y-%m-%d, nocase)
"S11"="2025-01-15";
fputk(Q\$SUBJ, Q\$VISIT, format="svdtn")

Returns native Date — arithmetic works
fmap() for data · fparse() for text defs

FPUT_DF() — FORMAT DATA FRAME COLUMNS

fput_df(data, ..., suffix="_fmt", replace=FALSE)
... = col=format pairs | suffix = new column suffix | replace = overwrite original

```
fput_df(df,
  sex = format_get("sex"),
  age = format_get("age"),
  suffix = "_lbl"
)
```

SEX	AGE	SEX_LBL	AGE_LBL
M	15	Male	Child
F	25	Female	Adult
NA	35	Unknown	Adult

MISSING VALUE HANDLING

#	CONDITION	RESULT
1	NA / NaN / ""	.missing label
2	Exact match	Mapped label
3	Range match	Range label
4	No match	.other / original

IS_MISSING() — CHECK FOR MISSING VALUES

is_missing(x)
Returns TRUE for NA, NaN, and empty string ""

```
is_missing(NA) # TRUE
is_missing(NaN) # TRUE
is_missing("") # TRUE
is_missing("x") # FALSE
```

TRUE
TRUE
TRUE
FALSE

KEEP_NA OPTION

```
fput(c("M", NA), "sex")
fput(c("M", NA), "sex",
  keep_na = TRUE)
```

"Male" "Unknown"
"Male" NA

TEXT PARSING & NUMERIC RANGES

FPARSE() — PARSE VALUE/INVALUE TEXT

fparse(text=NULL, file=NULL)
text = format definition string | file = path to .txt file with definitions

```
fparse(text = '
VALUE age (numeric)
  [0, 18) = "Child"
  [18, 65) = "Adult"
  [65, HIGH] = "Senior"
  .missing = "Age Unknown"
;')
fputn(c(5,18,65,NA), "age")
```

"Child" "Adult" "Senior" "Age Unknown"

RANGE SYNTAX REFERENCE

SYNTAX	MEANING	MATH
[a, b]	both inclusive	a ≤ x ≤ b
(a, b)	right exclusive	a ≤ x < b
[a, b)	left exclusive	a < x ≤ b
(a, b)	both exclusive	a < x < b
HIGH	↗ upper bound	
LOW	↘ lower bound	

BMI WITH DECIMALS + .MISSING

```
fparse(text = '
VALUE bmi (numeric)
  [0, 18.5) = "Underweight"
  [18.5, 25) = "Normal"
  [25, 30) = "Overweight"
  [30, HIGH] = "Obese"
  .missing = "No data"
;')
fputn(c(1234.5, 9.875, NA), "cash")
```

BMI	RESULT
16.2	Underweight
25.0	Overweight
35.1	Obese
NA	No data

NUMERIC PATTERNS — CONTINUOUS DISPLAY

fnew(pattern, type="numeric", .missing=..., name=...)
one %-spec, flags before width: \$ currency, % literal, , grouping = "%\$,2f" / "%10.0f" / "%1f%"

```
fnew("%$,2f", .missing = "-",
  type = "numeric", name = "cash")
```

"\$1,234.50" "\$9.88" "-"
grouping comma goes BEFORE width: %10.2f OK → %10.2f is an error

EXCLUSIVE / INCLUSIVE BOUNDS + .OTHER

```
fparse(text = '
VALUE score (numeric)
  (0, 50) = "Low"
  (50, 100) = "High"
  .other = "Out of range"
;')
fputn(c(1234.5, 9.875, NA), "cash")
```

SCORE	LABEL
0	Out of range
50	Low
51	High
101	Out of range

PARSE MULTIPLE FORMATS AT ONCE

```
fparse(text = '
VALUE race (character)
  "W"="White" "B"="Black"
  "A"="Asian"
  .missing = "Unknown"
;')
INVALUE race_inv
  "White"=1 "Black"=2
  "Asian"=3
;')
```

Registers both in library:
race VALUE (char)
race_inv INVALUE (num)

RANGE_SPEC() & FMAP_RANGES() — RANGE HELPERS

range_spec(low, high, label, inc_low=TRUE, inc_high=FALSE) · fmap_ranges(low, high, label, inc_low, inc_high, date_format=NULL)
range_spec describes ONE range (label required, returns a list) — fmap_ranges builds data-driven canonical keys for fnew()

```
# canonical keys for fnew():
fnew("0,18,TRUE,FALSE" =
  "Child", type = "numeric")
```

```
# From data arrays:
f <- fmap_ranges(
  low = c(0, 18), high = c(18, Inf),
  label = c("Child", "Adult"))
fputn(c(5, 30), f)
```

"Child" "Adult"
Do not pass range_spec() objects into fnew() — use canonical "low,high,inc_low,inc_high" keys or fmap_ranges()

REVERSE FORMATTING (INVALUE)

FINPUTN() — LABELS → NUMERIC

finputn(x, invalue_name)
Applies named invalue, returns numeric vector

```
finputn(
  c("Male", "Female", "Unknown"),
  "sex_inv"
)
```

1 2 NA

FINPUTC() — LABELS → CHARACTER

finputc(x, invalue_name)
Applies named invalue, returns character vector

```
finputc(
  c("Active", "Inactive"),
  "status_inv"
)
```

"A" "I"

FINPUTK() — COMPOSITE REVERSE

finputk(..., invalue_name, sep="|", na_as_string=FALSE)
Labels pasted into a composite key — code components (mirror of fputk)

```
fnew_bid("A1" = "Alpha",
  "B2" = "Beta", name = "pair")
```

"A1" "B2"

```
finputk(c("Alpha", "Beta"),
  invalue_name = "pair_inv")
```

fnew_bid registers the INVALUE as name_inv

ROUND-TRIP: VALUE ↔ INVALUE

"A" → fputc "status" → "Active" → finputc "status_inv" → "A"

DATE, TIME & DATETIME

SAS DATE FORMATS (AUTO-RESOLVED)

```
fputn(Sys.Date(), "DATE9.")
fputn(Sys.Date(), "MMDDYY10.")
fputn(Sys.Date(), "YYMMDD10.")
fputn(Sys.Date(), "WORDDATE.")
fputn(Sys.Date(), "QTR.")
```

FORMAT	→ RESULT
DATE9.	18MAR2026
MMDDYY10.	03/18/2026
YYMMDD10.	2026-03-18
WORDDATE.	March 18, 2026
QTR.	1

TIME FORMATS (SECONDS SINCE MIDNIGHT)

SECS	TIME8.	TIME5.	HHMM.
0	0:00:00	0:00	00:00
3600	1:00:00	1:00	01:00
45000	12:30:00	12:30	12:30
86399	23:59:59	23:59	23:59

DATETIME FORMATS

```
fputn(Sys.time(),
  "DATETIME20.")
```

FORMAT	→ RESULT
DATETIME20.	18MAR2026:13:48:58
DATETIME13.	18MAR26:13:48
DTDATE.	18MAR2026
DTYYMMDD.	2026-03-18

CUSTOM DATE FORMAT + .MISSING

```
v <- fnew_date("DATE9.",
  name = "vfmt",
  .missing = "NOT RECORDED")
fput_df(patients,
  visit_date = v)
```

ID	VISIT_DATE	VISIT_FMT
1	2025-01-10	10JAN2025
2	2025-02-15	15FEB2025
3	<NA>	NOT RECORDED

PUTN WORKFLOW (SAS-STYLE DYNAMIC DISPATCH)

```
fnew(
  "1" = "date9.",
  "2" = "mmddyy10.",
  name = "wfmt",
  type = "numeric")
datefmt <- fputn(key, "wfmt")
date <- fputn(number, datefmt)
```

NUM	KEY	FMT	DATE
12103	1	date9.	20FEB2003
10899	2	mmddyy10.	11/04/1999

FORMAT LIBRARY

FPRINT() — INSPECT REGISTERED FORMATS

```
fprint(name=NULL)
No args = list all | name = show detail for one format
fprint() # list all
fprint("sex") # detail
```

age	VALUE	(num)
sex <th>VALUE</th> <th>(char)</th>	VALUE	(char)
sex_inv <th>VALUE</th> <th>(num)</th>	VALUE	(num)

FORMAT_GET() FCLEAR()

format_get(name) — retrieve format object fclear(name=NULL) — remove one or all

```
fmt <- format_get("sex")
fclear("sex") # remove one
fclear() # clear all
```

Retrieve format object
Remove single format
Clear entire library

FLEVELS() FRANGES() FLIST() — INTROSPECT

flevels(fmt) — tidy value-label data.frame (range keys canonicalized) franges(fmt) — range rows flist() — registered names

	VALUE	LABEL
M	Male	
F	Female	

FEXPORT() — EXPORT TO PARSEABLE TEXT

fexport(..., formats=NULL, file=NULL)
... or formats = format objects | file = write to file (else returns string)

```
cat(fexport(bmi = bmi_fmt))
```

VALUE bmi (numeric)
[0, 18.5) = "Underweight"
[18.5, 25) = "Normal"
[25, 30) = "Overweight"
[30, HIGH] = "Obese"
;

FIMPORT() — SAS CNTLOUT CSV

fimport(file, register=TRUE, overwrite=TRUE)
file = CNTLOUT CSV path | register = auto-add to library | overwrite = replace existing

```
x <- fimport("cntlout.csv")
m <- fimport(csv,
  register = FALSE)
```

AGEGRP	VALUE	(num)
BMICAT	VALUE	(num)
GENDER	VALUE	(char)

```
fput(v, m[["GENDER"]])
```

0.8.5

Vladimir Larchenko · GPL-3
install.packages("ksformat")

MULTILABEL FORMATS

FPUT_ALL() — ONE VALUE → MULTIPLE LABELS MULTILABEL=TRUE

fput_all(x, format, ..., keep_na=FALSE)
Returns list of character vectors — all matching labels per value

```
fnew(
  "0,5,T,T" = "Infant",
  "6,11,T,T" = "Child",
  "12,17,T,T" = "Adolescent",
  "0,17,T,T" = "Pediatric",
  "18,64,T,T" = "Adult",
  "65,Inf,T,T" = "Elderly",
  "18,Inf,T,T" = "Non-Pediatric",
  name = "age_cat",
  type = "numeric",
  multilabel = TRUE
)
```

3 → Infant Pediatric
14 → Adolescent Pediatric
25 → Adult Non-Pediatric
70 → Elderly Non-Pediatric

fput() → first match only · fput_all() → all matching labels (list)

```
fput_all(
  c(3, 14, 25, 70),
  "age_cat"
)
```

AE SEVERITY GRADING (CLINICAL)

```
fnew(
  "1,1,T,T" = "Mild",
  "2,2,T,T" = "Moderate",
  "3,3,T,T" = "Severe",
  "4,4,T,T" = "Life-threat.",
  "5,5,T,T" = "Fatal",
  "5,5,T,T" = "Serious",
  "1,2,T,T" = "Non-serious",
  name = "ae",
  type = "numeric",
  multilabel = TRUE
)
```

1 → Mild Non-serious
2 → Moderate Non-serious
3 → Severe Serious
4 → Life-threat. Serious
5 → Fatal Serious

EXPRESSION LABELS

SPRINT WITH .X1 (EVALUATED AT APPLY-TIME)

```
stat <- fnew(
  "n" = "sprintf('%s', .x1)",
  "pct" = "sprintf('%1f%',
    .x1 * 100)",
  name = "stat"
)
```

"42" "5.3%" "100" "25.5%"

```
fput(
  c("n", "pct", "n", "pct"),
  stat,
  c(42, .053, 100, .255)
)
```

E() — FORCE EVALUATION WITHOUT .XN

```
lbl <- fnew(
  "ts" = e("format(Sys.Date(), '%Y')"),
  name = "demo"
)
```

"2026"
labels are VERBATIM unless they contain .xN or are wrapped in e() — failed eval = warning + NA, not an error

TWO ARGS (.X1, .X2) + SCALAR RECYCLING

```
r <- fnew(
  "ratio" = "sprintf('%s/%s',
    .x1, .x2)"
)
```

fput("ratio", r, 3, 10)

"3/10"
"42(N=100)" "55(N=100)"
.x2=100 recycled for all elements

FPUT(C("val", "val"), lbl,

```
c(42, 55), 100)
```

VECTORIZED FORMAT NAMES

EACH ELEMENT USES A DIFFERENT FORMAT (SAS PUTC/PUTN)

```
fnew(
  "1" = "groupx",
  "2" = "groupy",
  "3" = "groupz",
  name = "typefmt",
  type = "numeric"
)
```

TYPE	RESP	FMT	WORD
1	positive	groupx	agree
1	negative	groupx	disagree
2	positive	groupy	accept
2	negative	groupy	reject

define groupx, y, z...
respfmt <- fput(type, "typefmt")
word <- fputc(response, respfmt)

FUNCTION REFERENCE

CATEGORY	FUNCTION	PURPOSE
Create	fnew(..., name)	Discrete value→label
	fmap(keys, values)	Data-driven key→value
	finput(..., name)	Reverse label→value
	fnew_bid(..., name)	Both format + invalue
	fnew_date(pattern, name)	Date/time/datetime
	fmap_ranges/fmap_strata()	Range/stratified keys
Apply	fput(x, format, ...)	Generic apply
	fputn(x, name, ...)	Numeric apply
	fputc(x, name, ...)	Character apply
	fputk(..., format, sep)	Composite key apply
	fput_all(x, format)	All labels (multilabel)
	fput_df(data, ...)	Data frame columns
Reverse	finputn(x, name)	Labels → numeric
	finputc(x, name)	Labels → character
	finputk(..., invalue_name)	Composite reverse
Library	format_get(name)	Get from library
	fprint(name)	List/inspect
	fclear(name)	Remove format(s)
	fimport(file)	SAS CNTLOUT CSV
	flevels(fmt)	Value/label frame
	franges(fmt)	Range audit rows
	flist()	Registered names
Utils	fparse(text, file)	Parse definitions
	fexport(..., file)	Export to text
	range_spec(lo, hi, label)	One range descriptor
	e(expr)	Force label eval
Check	is_missing(x)	NA/NaN/"" → TRUE